

Question 1

Correct

Marked out of 1.00

Flag question

Question text

A binary number is a combination of 1s and 0s. Its n^{th} least significant digit is the n^{th} digit starting from the right starting with 1. Given a decimal number, convert it to binary and determine the value of the the 4^{th} least significant digit.

Example

number = 23

- Convert the decimal number 23 to binary number: $23_{10} = 2_4 + 2_2 + 2_1 + 2_0 = (10111)_2$.
- The value of the 4^{th} index from the right in the binary representation is 0.

Function Description

Complete the function fourthBit in the editor below.

fourthBit has the following parameter(s):

int number: a decimal integer

Returns:

int: an integer 0 or 1 matching the 4th least significant digit in the binary representation of number.

Constraints

$$0 \leq \text{number} < 2_{31}$$

Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the function.

103

The only line contains an integer, number.

Sample Case 0

Sample Input 0

STDIN Function

32 → number = 32

Sample Output 0

0

Explanation 0

- Convert the decimal number 32 to binary number: $32_{10} = (100000)_2$.
- The value of the 4th index from the right in the binary representation is 0.

Sample Case 1

Sample Input 1

STDIN Function

77 → number = 77

Sample Output 1

1

Explanation 1

- Convert the decimal number 77 to binary number: $77_{10} = (1001101)_2$.
- The value of the 4th index from the right in the binary representation is 1.

Answer:(penalty regime: 0 %)

```
1 int fourthBit(int number)
2
3 {
4
5     int i=4,sum;
6     while(i-->0)
7     {
8         if(number%2==0)
9             sum+=0;
10        else
11            sum+=1;
12        number=number/2;
13    }
14    return sum;
15 }
```

	Test	Expected	Got	
✓	printf("%d", fourthBit(32))	0	0	✓
✓	printf("%d", fourthBit(77))	1	1	✓

Passed all tests! ✓

Question 2

Correct

Marked out of 1.00

Flag question

Question text

Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the p_{th} element of the list, sorted ascending. If there is no p_{th} element, return 0.

Example

$n = 20$

$p = 3$

The factors of 20 in ascending order are {1, 2, 4, 5, 10, 20}. Using 1-based indexing, if $p = 3$, then 4 is returned. If $p > 6$, 0 would be returned.

Function Description

Complete the function `pthFactor` in the editor below.

`pthFactor` has the following parameter(s):

`int n`: the integer whose factors are to be found

`int p`: the index of the factor to be returned

Returns:

`int`: the long integer value of the p_{th} integer factor of n or, if there is no factor at that index, then 0 is returned

Constraints

$1 \leq n \leq 10_{15}$

$1 \leq p \leq 10_9$

Input Format for Custom Testing

Input from `stdin` will be processed as follows and passed to the function.

The first line contains an integer n , the number to factor.

The second line contains an integer p , the 1-based index of the factor to return.

Sample Case 0

Sample Input 0

STDIN Function

10 → n = 10

3 → p = 3

Sample Output 0

5

Explanation 0

Factoring n = 10 results in {1, 2, 5, 10}. Return the p = 3_{rd} factor, 5, as the answer.

Sample Case 1

Sample Input 1

STDIN Function

10 → n = 10

5 → p = 5

106

Sample Output 1

0

Explanation 1

Factoring n = 10 results in {1, 2, 5, 10}. There are only 4 factors and p = 5, therefore 0 is returned as the answer.

Sample Case 2

Sample Input 2

STDIN Function

1 → n = 1

1 → p = 1

Sample Output 2

1

Explanation 2

Factoring n = 1 results in {1}. The p = 1_{st} factor of 1 is returned as the answer.

Answer:(penalty regime: 0 %)

```

1 long pthFactor(long n,long p)
2
3 {
4
5     int count=0,i=1;
6     while(count!=p)
7     {
8         if(n%i==0)
9             count++;
10        else if(i>n)
11            return 0;
12        i++;
13    }
14    return i-1;
15 }

```

	Test	Expected	Got	
✓	printf("%ld", pthFactor(10, 3))	5	5	✓
✓	printf("%ld", pthFactor(10, 5))	0	0	✓
✓	printf("%ld", pthFactor(1, 1))	1	1	✓

Passed all tests! ✓

Question 1

Correct Marked out of 1.00 Flag question

You are a bank account hacker. Initially you have 1 rupee in your account, and you want exactly N rupees in your account. You wrote two hacks, first hack can multiply the amount of money you own by 10, while the second can multiply it by 20. These hacks can be used any number of time. Can you achieve the desired amount N using these hacks. Constraints: $1 \leq T \leq 100$ $1 \leq N \leq 10^{12}$ Input · The test case contains a single integer N. Output Quiz navigation Show one page at a time Finish review 1 2 REC-CIS For each test case, print a single line containing the string "1" if you can make exactly N rupees or "0" otherwise. SAMPLE INPUT 1 SAMPLE OUTPUT 1 SAMPLE INPUT 2 SAMPLE OUTPUT 0

Answer: (penalty regime: 0 %)

```
1 int myFunc(int n)
2 {
3     if (n == 1)
4         return 1;
5     if (n % 10 == 0)
6         if (myFunc(n / 10) == 1)
7             return 1;
8     if (n % 20 == 0)
9         if (myFunc(n / 20) == 1)
10            return 1;
11    return 0;
12 }
```

	Test	Expected	Got	
✓	printf("%d", myFunc(1))	1	1	✓
✓	printf("%d", myFunc(2))	0	0	✓
✓	printf("%d", myFunc(10))	1	1	✓
✓	printf("%d", myFunc(25))	0	0	✓
✓	printf("%d", myFunc(200))	1	1	✓

Passed all tests! ✓

Question 2

Correct Marked out of 1.00 Flag question

```
if (myFunc(n / 20) == 1) return 1; return 0; } Test Expected Got  
printf("%d", myFunc(1)) 1 1 printf("%d", myFunc(2)) 0 0  
printf("%d", myFunc(10)) 1 1 printf("%d", myFunc(25)) 0 0  
printf("%d", myFunc(200)) 1 1 Passed all tests! Find the number  
of ways that a given integer, X, can be expressed as the sum of  
the N powers of unique, natural numbers. For example, if X =  
13 and N = 2, we have to find all combinations of unique  
squares adding up to 13. The only solution is 2 + 3 . th 2 2 9 10  
11 12 REC-CIS Function Description Complete the powerSum  
function in the editor below. It should return an integer that  
represents the number of possible combinations. powerSum  
has the following parameter(s): X: the integer to sum to N: the  
integer power to raise numbers to Input Format The first line  
contains an integer X. The second line contains an integer N.  
Constraints  $1 \leq X \leq 1000$   $2 \leq N \leq 10$  Output Format Output a  
single integer, the number of possible combinations calculated.  
Sample Input 0 10 REC-CIS 2 Sample Output 0 1 Explanation 0 If  
X = 10 and N = 2, we need to find the number of ways that 10  
can be represented as the sum of squares of unique numbers.  
10 = 1 + 3 This is the only way in which 10 can be expressed as  
the sum of unique squares. Sample Input 1 100 2 Sample  
Output 1 3 Explanation 1 2 2 REC-CIS 100 = (10 ) = (6 + 8 ) = (1 +  
3 + 4 + 5 + 7 ) Sample Input 2 100 3 Sample Output 2 1  
Explanation 2 100 can be expressed as the sum of the cubes of  
1, 2, 3, 4. (1 + 8 + 27 + 64 = 100). There is no other way to  
express 100 as the sum of cubes. Answer: (penalty regime: 0 %)
```

```

1 int powerSum(int x, int m, int n)
2 {
3     int tmp;
4     tmp = 1;
5     for (int i = 1; i <= n; i++)
6     {
7         tmp = tmp * m;
8     }
9     if (tmp == x)
10        return 1;
11    if (tmp > x)
12        return 0;
13    return powerSum(x, m + 1, n) + powerSum(x - tmp, m + 1, n);
14 }

```

	Test	Expected	Got	
✓	printf("%d", powerSum(10, 1, 2))	1	1	✓

Passed all tests! ✓